ac^{2}SLAM: FPGA Accelerated High-Accuracy SLAM with Heapsort and Parallel Keypoint Extractor

Cheng Wang[1], Yingkun Liu[1], Kedai Zuo[1], Jianming Tong[2], Yan Ding[1] and Pengju Ren[1]

[1]*Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University*
[2]*College of Compute, Georgia Institute of Technology*

2021 International Conference on Field-Programmable Technology (ICFPT) | 978-1-6654-2010-5/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICFPT52863.2021.9609808

***Abstract*—In order to fulfill the rich functions of the application layer, robust and accurate Simultaneous Localization and Mapping (SLAM) technique is very critical for robotics. However, due to the lack of sufficient computing power and storage capacity, it is challenging to delpoy high-accuracy SLAM in embedded devices efficiently. In this work, we propose a complete acceleration scheme, termed *ac^{2}SLAM*, based on the ORB-SLAM2 algorithm including both front and back ends, and implement it on an FPGA platform. Specifically, the proposed *ac^{2}SLAM* features with: 1) a scalable and parallel ORB extractor to extract sufficient keypoints and scores for throughput matching with 4% error, 2) a Ping-Pong heapsort component (PP-HEAPSORT) to select the significant keypoints, that could achieve single-cycle initiation interval to reduce the amount of data transfer between accelerator and the host CPU, and 3) the potential parallel acceleration strategies for the back-end optimization. Compared with running ORB-SLAM2 on the ARM processor, *ac^{2}SLAM* achieves 2.1× and 2.7× faster in the TUM and KITTI datasets, while maintaining 10% error of SOTA eSLAM. In addition, the FPGA accelerated front-end achieves 4.55× and 40× faster than eSLAM and ARM. The ac^{2}SLAM is fully open-sourced at *https://github.com/SLAM-Hardware/acSLAM*.**

I. Introduction

Simultaneous Localization and Mapping (SLAM) is the algorithm for constructing a map and keeping tracking robot localization in unknown surroundings, which is critical for acting as the necessary underlying component to support high-level applications, like navigation and multi-agent collaboration. When it comes to deployment on autonomous mobile robots, constrained by the stringent power budget and physical space, the compute power of embedded CPU is insufficient to serve the SLAM algorithms in real-time. Hence, the design of a dedicated accelerator has attracted widespread attention from academia and industry.

ORB-SLAM is a widely applied robust feature-based SLAM approach that estimates robot poses by matching the extracted ORB features [1]. The structure of ORB-SLAM is shown in Fig. 1. The ORB features are extracted through oriented Feature from Accelerated Segment Test (oFAST) [2] feature detection and Binary Robust Independent Elementary (BRIEF) [3] descriptor computing. Because the feature extraction is the most time-consuming stage in the front-end, current SLAM accelerators mainly focus on the front-end acceleration while simplifying the back-end with sacrifice of accuracy [4]–[8], rendering inferior performance in the large open environment, especially scenarios with multiple loops. Therefore, mobile robots need *a complete SLAM system with accuracy-sufficient optimization to support its real-time deployment in large scenarios.*

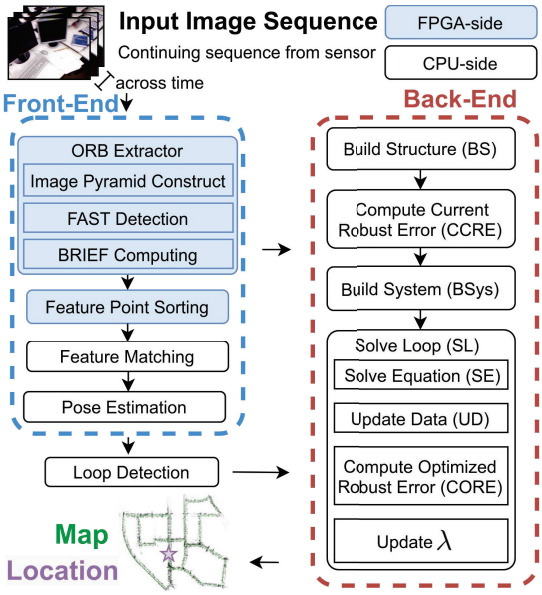


Fig. 1: Structure of the ORB-SLAM

The front-end could extract thousands of keypoints per image. However, only significant keypoints are needed to be selected for further processing, thus, an efficient hardware-based keypoints extracting and sorting architecture is necessary to speedup the execution and filter the keypoints to save both time and transferring bandwidth between CPU and accelerator. Moreover, the number of extracted keypoints varies with different input frames, and the dynamically changing keypoints set invalidate lots of sorting strategies [9]–[13]. Because most hardware sorting circuits deal with a fixed number of inputs, once the condition is not met, it will lead to either waste or shortage of hardware resources. This might even cause insufficient number of keypoint pairs between different frames and degrade the performance of the SLAM.

The hardware heap sorting [14] that can select top-K elements of any data size is suitable to solve this problem, but it still suffers from the multi-cycle initiation interval (II), because the unavoidable write-read latency of Block-RAM (BRAM) requires the newly written data could only be used after at least two cycles. Especially when dozens of front-end ORB extraction modules are executing simultaneously, this situation will be further exacerbated and makes the unoptimized write-read latency a new bottleneck. Alternatively, simply replacing BRAMs with Look-Up Table (LUT) will consume prohibitive resources that the embedded FPGA could provide. Thus, it's

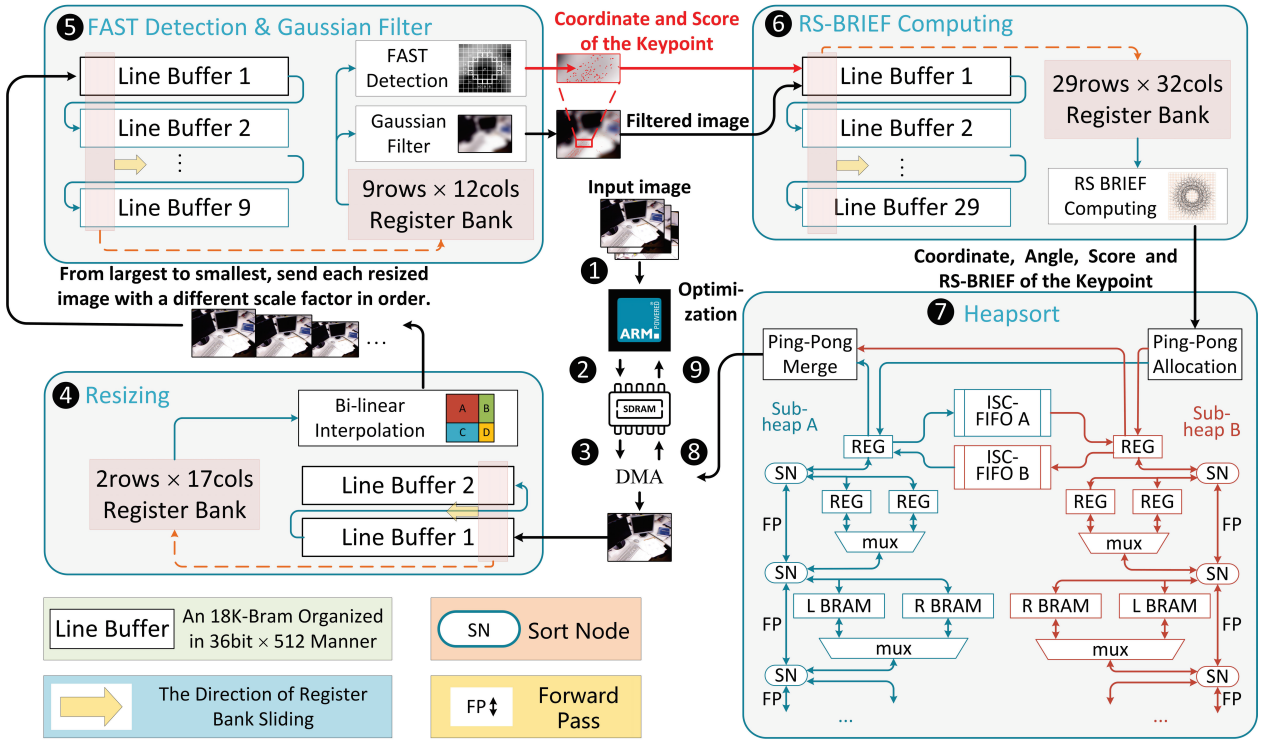


Fig. 2: Overall architecture of ac^{2}SLAM

demanding *an efficient hardware sorting architecture satisfying II=1 with reasonable resources, that could tackle the dynamic size of the input data.*

To achieve the above-mentioned goals, we make the following contributions:

- An open-source FPGA-based high-accuracy ORB-SLAM2 [15] scheme which supports monocular, stereo, and RGB-D cameras in large scenarios.
- A scalable ORB extractor hardware that could compute $4n$ pixels per cycle in parallel ($n = 1, 2, 3 \ldots$). Throughput matching among modules could be easily achieved through tuning the parallelism.
- PP-HEAPSORT with FIFO-based inter-heapsort coordination could circumvent the write-read latency restriction of BRAM and achieve a single-cycle initiation interval. And it only consumes 16% LUT and 20% FF resources of the design implemented without BRAM. Further, a heapsort with throughput of 2^n ($n = 1, 2, \ldots$)-data-per-cycle could be constructed with a tree structure of PP-HEAPSORTs.
- We analyze the bottlenecks in the back-end on the level of matrix operator and raise a strategy of accelerating operation by parallelizing matrix computation on the ZYNQ FPGA, which we believe could also inspire further acceleration work.

II. System Overview

The overall architecture of ac^{2}SLAM is shown in Fig. 2. We implement ORB extractor and heapsort as specialized hardware modules, and combine with the CPU to realize a heterogeneous end-to-end system for ORB-SLAM2. The overall workflow of the system is shown below:

❶, ❷ and ❸: The ARM processor receives frames from the camera or dataset and then stores them in off-chip memory, then reads and transfers images to the programmable logic on FPGA (PL-side) through Direct Memory Access (DMA).

❹: The scale invariance [16] for keypoint detection requires that the input image should be scaled down iteratively in the resizing module to get an 8-layer image pyramid. The scale ratio is 1.2 in each iteration. Thus the scale factor from the topmost layer to the bottom layer of the pyramid is about 3.5.

❺: FAST is a corner detection method that can detect the coordinate positions of keypoints in an image and Gaussian filter is used to reduce the noise. This module extracts FAST keypoints from each input image and scores them, while simultaneously blurring the image with Gaussian filter.

❻: RS-BRIEF Computing module calculates the angles and RS-BRIEF [4] descriptor of the keypoints. Afterwards, it shifts the RS-BRIEF according to the angle, and sends the coordinates, angles, scores and RS-BRIEFs of the keypoints to the downstream process.

❼: The heapsort module filters the keypoints to store $2^n - 2$ ($n = 1, 2, 3 \ldots$) elements with the highest scores. The challenge of this part is that the number of keypoints varies for different images, which needs an efficient hardware design that supports sorting of varying numbers of inputs.

❽: Selected keypoints with corresponding coordinates, angles and scores are transferred back to the CPU through DMA.

❾: The ARM processor reads results of step ❼ from the off-chip memory and performs features tracking to estimate the pose of the camera. Then, the back-end will be launched to reduce the cumulative error and noise of the estimated poses.

III. Front-End Implementation

This section will describe the architecture of the proposed scalable ORB extractor that could compute $4n$ pixels per cycle in parallel ($n = 1, 2, 3, \ldots$). All parts in the ORB extractor have

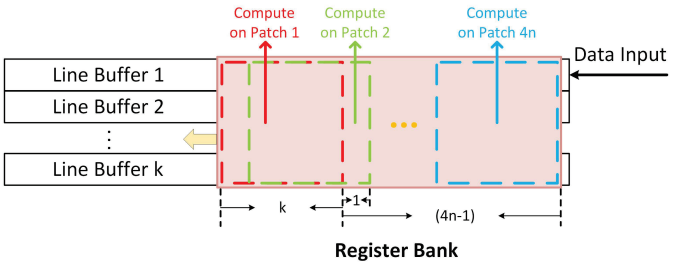


Fig. 3: The parallel register banks

1-cycle initiation interval for different n. All three modules including Resizing, FAST detection and RS-BRIEF computing share the same image patches. Parallel strategy is shown in Fig. 3. This section will first introduce the overall architecture, and then discuss the detailed implementation of each module.

A. ORB Feature Extractor

a) Line Buffer: An input image with column number N_c will need a BRAM with depth $2^{\lceil \log(N_c/(4n)) \rceil}$. Besides, a parallelism $4n$ will require the BRAM with width $4n \times 9$ bit. In our implementation for KITTI dataset, each line buffer comprises an 18K-BRAM organized in 36 bit $\times$ 512 manner, which can store one row of pixels of the input image.

b) Register Bank: We implement a large register bank as shown in Fig. 3. The $k \times (k + 4n - 1)$ register bank is divided into $4n$ k-by-k image patches. Each image patch performs a single operator, i.e. bi-linear interpolation operator in the resizing module, keypoint detection operator in FAST Detection module or RS-BRIEF operator in RS-BRIEF Computing module. Multiple operators are performed on different image patches in parallel. The whole register bank slides in the same direction as data is fed into the line buffer. The register bank slides by $4n$ pixels per clock cycle.

c) Operation Detail: Since keypoints are sparsely distributed in an image, i.e. the number of keypoints is generally no more than 1% of the number of all pixels in an image, we use $4n$ patches to determine the existence of keypoints in parallel. If finding a keypoint, the sliding register bank stops and $4n$ image patches inside the register bank are computed. Specifically, in Fig. 2, multiple image patches in ❹ and ❺ are calculated in parallel, while ❻ calculates image patches sequentially because parallel unfold will consume too much resources. Conversely, if no keypoint is encountered, the module simply skips the computation process and slides register bank to the next location to save time. We analyze the impact on ORB extractor's latency in Section VI.

B. FPGA Implementation of the Front-end Modules

a) Resizing: We use the bi-linear interpolation to resize the input image to different sizes, which are layers of the image pyramid. To ensure $4n$-pixel throughput under the largest scale factor 3.5, i.e. a single valid output appears every 3.5 image patches under the largest scale factor, the resizing requires $3.5 \times 4n \approx 16n$ images patches. Therefore, we implement a $2 \times (2 + 16n - 1)$-pixel register bank in bi-linear interpolation.

b) FAST Detection & Gaussian Filter: To detect whether a pixel is a keypoint, a 9×9 image patch is needed to implement FAST detection with Non-Maximum Suppression (NMS) on the center pixel. Expanding the pixel register bank from 9×9 to $9 \times (9 + 4n - 1)$ enables detection of $4n$ pixels in parallel.

c) RS-BRIEF Computing: We randomly generate a large number of RS-patterns by combining the method of generating patterns in [3] and the rules of RS-BRIEF in [4]. Then the one with the highest accuracy in TUM [17] and KITTI [18] datasets is chosen. Besides, the migration from BRIEF to RS-BRIEF will degrade the accuracy of DBoW2 model [19], which is used in re-localization, re-initialization and loop closing. Therefore, we train a new vocabulary with our RS-BRIEF pattern by TUM and KITTI datasets that can adapt to both indoor and outdoor environments. A single RS-BRIEF pattern requires 29×29 image patch, which requires a $29 \times (29 + 4n - 1)$ pixel register bank in the system to achieve $4n$ parallelism in theory. The computation logic for 29×29 image patch is so resource-demanding that only one of the computation logic is constructed to process all image patches of the register bank in sequential. Given the sparsity of the keypoints, the sequential processing doesn't degrade the performance in most cases.

IV. PP-Heapsort Implementation

The dynamic input size invalidates some classical hardware sorting architectures like bitonic-merge sorting, etc. On the other hand, existing sorting hardware that could adapt to the dynamic input size, like heapsort, either requires prohibitive LUT resources or comprises for multiple cycles initiation interval (II) because of the write-read latency restriction of BRAM. To eliminate the above issues, a low-resource-overhead PP-HEAPSORT implementation with II=1 is proposed in this section.

A. Functionality of a Typical Heapsort

A typical sub-heap is constructed by a heap of Sorting Node (SN) and Data Storage [14], shown as a sub-heap in the ❼ of Fig. 2. A n-level heapsort mainly performs two functionalities: (1) select the largest $2^n - 1$ values out of the consecutive input data sequence. The largest $2^n - 1$ values are termed as **top** $2^n - 1$ **largest data**. (2) maintain all data inside the heapsort in ascending order from top to the bottom, i.e. a small top heap structure. Therefore, the topmost data is always the **minimal value** of the whole heap. As for a typical heapsort with size of $2^n - 1$, new arriving data, if **larger** than the topmost data, will kick out the top data and take its place. Once top data gets replaced, the heapsort will adjust itself to put the new minimal data of the sub-heap at the top. The process enables the sub-heap to always maintain the top $2^n - 1$ values of the input data sequence of various size. Note that functionality of hardware heapsort is not similar as heapsort data structure, and detail functionality of a heapsort could be found at [14].

B. Why Ping-Pong and How Ping-Pong?

We use BRAM to implement heapsort, since BRAM has at least two-cycle write-read latency, the hardware heapsort based on BRAM requires at least two cycles of the initiation interval. Considering the worst case of comparison, two consecutive comparisons use the same data. In the first comparison in the first cycle, the data is moved to another location and could only be used after two cycles. When it comes to the second comparison in the second cycle, the data is not ready, leading

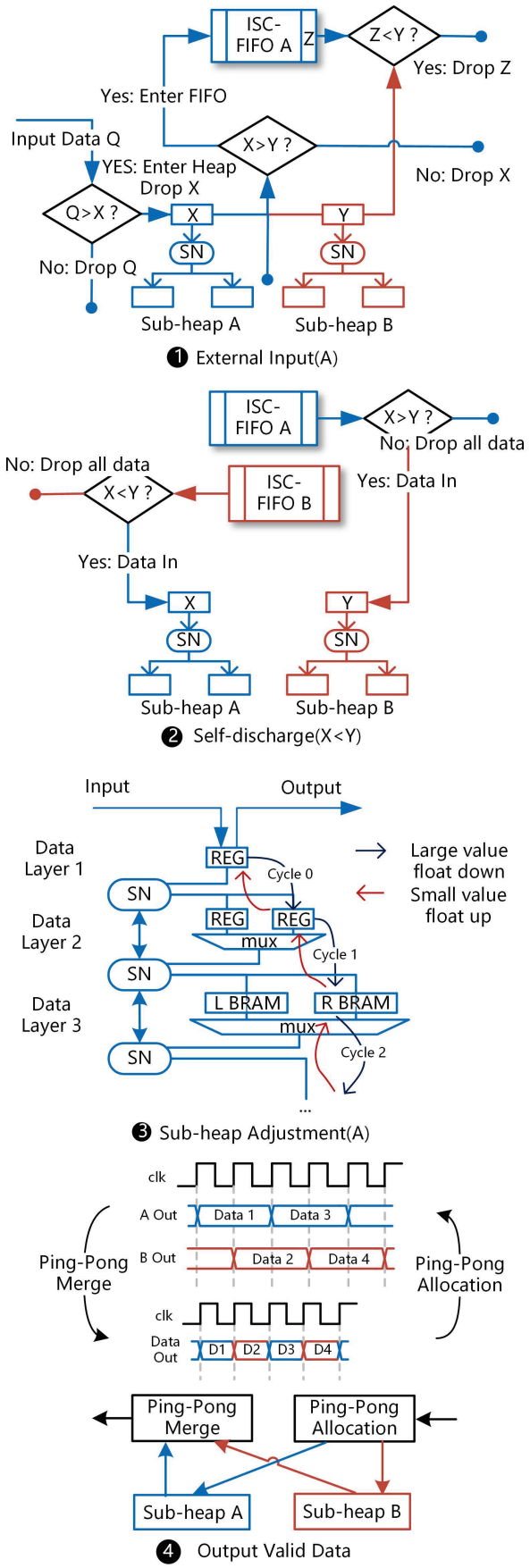


Fig. 4: PP-HEAPSORT work process

to a data hazard. *Instead of eliminating the two-cycle write-read latency* , we circumvent it by the Ping-Pong strategy. Specifically, by leveraging two sub-heaps with size $2^{(n-1)}-1$ and switching them in the Ping-Pong manner, as shown by ❹ in Fig 4, we ensure that each sub-heap receives data every two cycles to avoid data hazard. Further, the two Ping-Pong sub-heaps working as a whole could reach one-cycle initiation interval. We will illustrate three features of proposed PP-HEAPSORT implementation with size 2^n-2 in the following subsections.

C. Inter Sub-heaps Coordination FIFO (ISC-FIFO)

The PP-HEAPSORT architecture enables one-cycle initiation interval. But the two sub-heaps are performing sorting separately. Each of the two sub-heaps maintains its largest $2^{(n-1)}-1$ data individually. Notice that, it is possible that the data discarded from sub-heap A is smaller than all the data in sub-heap A, but it could be larger than some data in the sub-heap B, and vice versa. Such data is termed as coordinated data and should be kept instead of dropping. To avoid throwing away the coordinated data, a reasonable coordination mechanism needs to be designed. Specifically, we attach an Inter Sub-heaps Coordination FIFO (ISC-FIFO) for each sub-heap to store the data excluded from the corresponding sub-heap, and an additional maintenance process dividing the process of PP-HEAPSORT into the following four parts.

1) External Input Stage: New input data will be forwarded to one of the sub-heaps (sub-heap A in the following) by the Ping-Pong allocation as shown in ❶. After comparing with the top value of the sub-heap A, the larger one will be stored while the smaller one gets discharged (X in the following). In the same time, the top data Y of the sub-heap B, which is always the smallest value of the sub-heap B. Then, we need to make sure that X gets stored in ISC-FIFO B, if it is the coordinated data, and abandoned otherwise. Specifically, if $X > Y$, X is a coordinated data and should enter sub-heap B. Otherwise, X can be discarded directly. Considering that sub-heap B could only take data every 2 cycles, X cannot be directly fed into sub-heap B. Instead we store X into the ISC-FIFO A, and need corresponding logics to decide whether to re-enter X into sub-heap B or discard it.

Y will become larger as sub-heap B acquires more external data, so the data in ISC-FIFO A might be smaller than Y and the input data does not need to re-enter sub-heap B any more. Therefore, we compare the front data Z in ISC-FIFO A with Y, as shown in ❶ of Fig. 4. When $Z < Y$, Z will be directly dropped, the same method is used for ISC-FIFO B. This process continues until all external data have been fed into the PP-HEAPSORT.

2) ISC-FIFO Self-discharge Stage: After all the external data have been fed into the heapsort, all data in one of the ISC-FIFOs will be completed dropped because they are smaller than all data in both sub-heaps. This process is called ISC-FIFO self-discharge, as shown by ❷ in Fig. 4. For example, ISC-FIFO A only stores discharged data from sub-heap A, so that all the data in ISC-FIFO A will be not larger than X after external input stage. If $Y > X$, all the data in ISC-FIFO A will be not larger than any data in sub-heap B, so that data in ISC-FIFO

A don't need to be fed in sub-heap B and could be completely discharged. (Drop all data in ISC-FIFO B when $X > Y$.)

Simultaneously, the remaining ISC-FIFO B with data will start to feed its data to the sub-heap A to maintain the inter-subheap data coordination. This stage will be completed when the remaining ISC-FIFO B has sent out all the data.

3) Sub-heap Adjustment: When the ISC-FIFO self-discharge finished, we still need to wait for several cycles for the sub-heap to adjust inner data to keep the new minimal value at the top of the sub-heap. The latency of this process takes at most twice the number of level of a single sub-heap, as shown in ❸ in Fig. 4.

4) Output Valid Data: Both sub-heaps will start to output the inner value in the Ping-Pong manner once the Sub-heap Adjustment stage finishes, as shown by ❹ in Fig. 4.

D. Block RAM Reuse

At each cycle, there are at most two data with different addresses will be accessed in each level of the sub-heaps, which enables us to use a single sorting node and two BRAMs for each level [14].

On top of that, the level k of sub-heap will only need to store $2^{(k-1)}, k = 1, 2, ...$ number of data. When $k < 5$, the number of data to be stored does not exceed 20, which will waste lots of BRAM storage. Therefore, we replace the BRAM in the lower level with LUT to increase the overall utilization of BRAM.

E. Scale to 2^n Throughput

We illustrate a PP-HEAPSORT with single-data-per-cycle throughput in this section. Further, a heapsort with 2-data-per-cycle throughput could be constructed by arranging two PP-HEAPSORTs in parallel with two extra ISC-FIFOs to maintain the data coordination between them. And a generalized heapsort with 2^n $(n = 1, 2, \dots)$-data-per-cycle throughput could be built by constructing PP-HEAPSORTs in the tree structure with ISC-FIFOs between every two PP-HEAPSORTs sharing the same parent.

V. Back-end Analysis

A robust and high-accuracy back-end is another bottleneck in the SLAM system. In this section, we will firstly dig into G2O, the optimization tool used in back-end, to category patterns of operations, and then propose a parallel accelerator method to reduce the processing latency on multi-core CPU. In the end the proposed parallel acceleration will inspire potential accelerator design .

A. G2O in Back-end

The G2O is a graph-based nonlinear error function optimization tool. The graph of G2O consists of user-defined vertices and edges. In ORB-SLAM2, vertices include poses of cameras (v_c), typically a six-dimensional vector consisting of a three-dimensional rotation vector and a three-dimensional translation vector, and locations of landmarks (v_l), a three-dimensional coordinate. An edge (e) connecting v_c and v_l represents the re-projection error of the coordinate of the landmark (v_l) in the pixel coordinate system of the camera (v_c). The goal of optimization is to reduce the summation of re-projection errors of all edges $\sum e^2$ [20].

TABLE I: Symbol List of back-end

Meaning	Symbols
number of the edges	n_e
number of the camera vertices	n_c
number of the landmark vertices	n_l
number of the elements in camera vertices	$6n_c$
number of the elements in landmark vertices	$3n_l$
total number of vertices	$n_{cl} = n_l + n_c$
camera vertex, also 6-D pose of a camera	v_c
landmark vertex, also 3-D location of a landmark	v_l
all the vertices	$v = v_l + v_c$

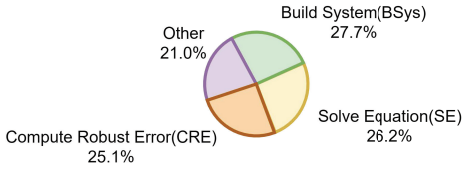


Fig. 5: Latency Breakdown of back-end

As shown in the Fig. 5, Build System and Solve Equation account for a large proportion of the overall latency in G2O-based back-end. The reason is analyzed in the following subsection V-B through detailed analysis of the computational complexity required by each module listed in the figure.

B. The Flow of G2O-based Back-end

A typical G2O optimization is to run an optimization flow for multiple iterations to gradually reduce the error. A single iteration of the optimization flow is shown below:

1) Build Structure (BS): This module converts the graph structure, including v_c, v_l and e, into the data structure needed to solve the incremental equation:

$$(\mathbf{H} + \lambda \cdot \mathbf{I}) \cdot x = b \tag{1}$$
$$where \quad \mathbf{H} = \mathbf{J^T} \cdot \mathbf{J} \quad b = -\mathbf{J^T} \cdot [e_1, \dots, e_{n_e}]^T$$

where $\mathbf{H}$ is a n_{cl}-by-n_{cl} block sparse matrix. All matrix blocks are indicated by bold format like $\mathbf{H}$. $\mathbf{J}$ is a n_e-by-n_{cl} sparse matrix which equals to $\frac{\partial e}{\partial v}$. λ is defined at the beginning and updated in every iteration. In this module, only memory space is allocated to $\mathbf{H}$ and b without any computation. Further, this module is only performed once on the first iteration, thus causing negligible latency in the overall back-end procedure.

2) Computing Current Robust Error (CCRE): This stage applies robust kernel to e, and then calculates $\sum e^2$ to obtain the $current\_robust\_error$. All edges should be accumulated together, which makes this step a computation overhead of n_e accumulation.

3) Build System (BSys): This stage calculates values of $\mathbf{H}$. $\mathbf{H}$ is typically divided into four parts by the relationship of vertices, as shown in the equation 2.

$$\mathbf{H} = \begin{bmatrix} \mathbf{B} & \mathbf{E} \\ \mathbf{E^T} & \mathbf{C} \end{bmatrix} \tag{2}$$
$$\mathbf{B} = \mathbf{J_c^T} \cdot \mathbf{J_c} \quad \mathbf{C} = \mathbf{J_l^T} \cdot \mathbf{J_l} \quad \mathbf{E} = \mathbf{J_c^T} \cdot \mathbf{J_l}$$
$$\mathbf{J_c} = \frac{\partial e}{\partial v_c} \quad \mathbf{J_l} = \frac{\partial e}{\partial v_l}$$

There are only edges e connecting the camera vertex v_c and the landmark vertex v_l and no intra-camera edges nor intra-landmark edges exist. Therefore, $\mathbf{B}$, $\mathbf{C}$ and $\mathbf{E}$ are n_c-by-n_c, n_l-by-n_l and n_c-by-n_l block diagonal matrices, respectively. Even

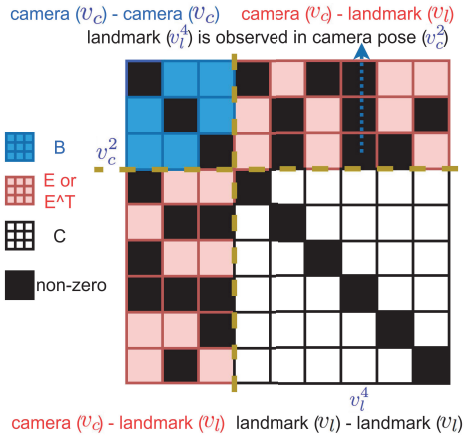


Fig. 6: The sparsity of $\mathbf{H}$

though $\mathbf{B}, \mathbf{C}, \mathbf{E}$ are all matrices with large dimensions, the sparse essential shown in Fig. 6 enables G2O to only calculate the non-zero elements. Thus G2O only needs to iterate the total number of edges instead of the number of elements inside matrix $\mathbf{H}$. In the i-th iteration, $\frac{\partial e_i}{\partial v_c^m}$ and $\frac{\partial e_i}{\partial v_l^n}$ are calculated, and $\mathbf{B}$, $\mathbf{C}$, $\mathbf{E}$ will get calculated as shown in the equation 3:

$$\mathbf{B_{mm}} += (\frac{\partial e_i}{\partial v_c^m})^T \cdot \frac{\partial e_i}{\partial v_c^m} \tag{3}$$

$$\mathbf{C_{nn}} += (\frac{\partial e_i}{\partial v_l^n})^T \cdot \frac{\partial e_i}{\partial v_l^n}$$

$$\mathbf{E_{mn}} += (\frac{\partial e_i}{\partial v_c^m})^T \cdot \frac{\partial e_i}{\partial v_l^n}$$

The meanings of above symbols are shown in Table II.

4) Solve Loop (SL): The Solve Loop module consists of Solve Equation, Update Data, Compute Optimized Robust Error and Update λ.

a) Solve Equation (SE): This step simplifies the incremental equation (formula 1) to the schur incremental equation (formula 4). The Schur Elimination reduces the scale of the incremental equation from n_{cl} to n_c:

$$[\mathbf{B} - \mathbf{E} \cdot \mathbf{C^{-1}} \cdot \mathbf{E^T}] \cdot x_c = u - \mathbf{E} \cdot \mathbf{C^{-1}} \cdot w \tag{4}$$

u is the first $6n_c$ elements of vector b, while w is the last $3n_l$ elements of vector b in equation 1. The block diagonal essential of matrix $\mathbf{C}$ will simplify the inversion into the number of n_l 3-by-3 matrix inversion. Even though the dimension of $\mathbf{E}$ in equation 4 is large, it will be divided into multiple small blocks which are calculated as shown below:

1) Pick n-th column, and select $\mathbf{E_n}$ and $\mathbf{C_{nn}^{-1}}$. The n will be iterated for the number of landmarks n_l.
2) All rows in this selected column $\mathbf{E_n}$, where the data block is non-zero, will be iterated in this step.
   - Pick m-th row. And thus select a small dense matrix block $\mathbf{E_{mn}}$. Then calculate $\mathbf{E_{mn}} \cdot \mathbf{C_{nn}^{-1}}$.
   - Pick k-th row, and select a small dense matrix block $\mathbf{E_{kn}}$. And then calculate

$$(\mathbf{E} \cdot \mathbf{C^{-1}} \cdot \mathbf{E^T})_{\mathbf{mk}} += \mathbf{E_{mn}} \cdot \mathbf{C_{nn}^{-1}} \cdot (\mathbf{E_{kn}})^T \tag{5}$$

Obtaining x_c from equation 4, x_l could be calculated:

TABLE II: Division of $\mathbf{B}, \mathbf{C}, \mathbf{E}$

Meaning	Symbols
the m-th row and the m-th column matrix block in $\mathbf{B}$	$\mathbf{B_{mm}}$
the n-th row and the n-th column matrix block in $\mathbf{C}$	$\mathbf{C_{nn}}$
the m-th row and the n-th column matrix block in $\mathbf{E}$	$\mathbf{E_{mn}}$
the entire n-th column of matrix $\mathbf{E}$	$\mathbf{E_n}$
the i-th edge	e_i
the m-th camera vertex	v_c^m
the n-th landmark vertex	v_l^n

$$x_l = \mathbf{C^{-1}} \cdot (w - \mathbf{E^T} \cdot x_c) \tag{6}$$

The dimension of x_c is usually several or dozens for Local BA and hundreds for Global BA. The small dimension of x_c together with the block diagonal feature of $\mathbf{C}$ reduces the number of calculation into the same magnitude with n_{cl}.

b) Update Data (UD): After obtaining x_c and x_l, Δ is used to term the union of x_c and x_l. And then update the data of the cameras and the landmarks: $v \leftarrow v + \Delta$.

c) Compute Optimized Robust Error (CORE): It leverages the same way to compute $optimized\_robust\_error$ as mentioned in CCRE with the updated camera poses and landmark locations. Then CRE is used to term the union of CCRE and CORE.

d) Update λ: This step calculates the temporal variable, named as ρ, which is required to update λ:

$$\rho = \frac{current\_robust\_error - optimized\_robust\_error}{\Delta \cdot (\lambda \cdot \Delta + b)} \tag{7}$$

The above 4 sub-functions in SL will be executed iteratively until $\rho > 0$ or the number of iterations hits the pre-defined threshold.

C. Parallelism Acceleration Analysis

With the computation analyzed in the previous subsection, we list our strategy of parallel acceleration here. The equation of matrix-block based calculation indicates that only output dependency exists for the two parts, because the results of multiple matrix-block operations should be accumulated together into the final result as following:

a) BSys: Equation 3 shows that $\mathbf{B_{mm}}, \mathbf{C_{nn}}$ and $\mathbf{E_{mn}}$ are the partial results that should be accumulated together, which introduces the output dependency. Even though the output dependency exists, speed-up could also be achieved by parallelizing the partial derivative and matrix multiplication calculation with only results accumulation performed in serial. Specifically, $\frac{\partial e_i}{\partial v_c^m}$, $\frac{\partial e_i}{\partial v_l^n}$, $(\frac{\partial e_i}{\partial v_c^m})^T \cdot \frac{\partial e_i}{\partial v_c^m}$, $(\frac{\partial e_i}{\partial v_l^n})^T \cdot \frac{\partial e_i}{\partial v_l^n}$ and $(\frac{\partial e_i}{\partial v_c^m})^T \cdot \frac{\partial e_i}{\partial v_l^n}$ can be conducted in parallel while the accumulation of $\mathbf{B_{mm}}$, $\mathbf{C_{nn}}$, $\mathbf{E_{mn}}$ is calculated in serial by using mutex to guarantee the correctness of the results.

b) SE: The number of row-iterations will not exceed n_c, and the sparse essential of matrix block further reduces the number of iterations in row dimension. Besides the number of iterations in column dimension equals to n_l, which has more potential of parallelism compared with row iterations and becomes our target for parallelization. The output dependency of different col-iteration lies in accumulation of multiple $(\mathbf{E} \cdot \mathbf{C^{-1}} \cdot \mathbf{E^T})_{mk}$. The same strategy could be applied here as

TABLE III: Utilization comparison of ac^{2}SLAM and eSLAM

	LUT	FF	DSP	BRAM
ac^{2}SLAM	146572	74166	173	61
eSLAM	56954	67809	111	78

TABLE IV: The Latency of Feature Extraction

	TUM (640×480)	KITTI(1241×376)
ac^{2}SLAM	2.0 ms	5.1 ms
eSLAM	9.1 ms	not apply
ARM	80.2 ms	202.7 ms

well to keep the accumulation of $(\mathbf{E} \cdot \mathbf{C^{-1}} \cdot \mathbf{E^T})_{mk}$ in serial with $\mathbf{E_{mn}} \cdot \mathbf{C_{nn}^{-1}} \cdot (\mathbf{E_{kn}})^T$ calculated in parallel.

To make a conclusion, the back-end optimization could be divided into parallel multiple matrix operations, which could be accelerated by (1) parallelizing both BSys and SE, and (2) accumulating results of paralleled matrix operations together.

VI. Experiment

In this section, we deploy the proposed hardware design on FPGA, and analyze the whole system and each proposed module.

A. Hardware-software Co-designed System Experiment

1) Latency-resource ratio analysis: We implement ac^{2}SLAM on a Zynq® UltraScale+™ MPSoC XCZU7EV device, which integrates a quad core ARM® Cortex™-A53 processing system (PS) and a dual-core ARM Cortex-R5 real-time processor. The CPU frequency of PS is 1.5GHz, and the clock frequency of the accelerating modules is 100 MHz. We demonstrate the deployment of a keypoint extractor with 4 parallelism, i.e., configure proposed hardware with $n = 1$ in Fig. 2. The comparison of the resource utilization between ac^{2}SLAM and eSLAM [4] is shown in Table III.

The SLAM accelerator system is evaluated on the TUM and KITTI datasets. The latency of feature extraction in 4-level image pyramid for TUM and 8-level image pyramid for KITTI is shown in Table IV. Compared with eSLAM, ac^{2}SLAM uses only $2.6\times$ LUTs, $1.1\times$ FFs, $1.56\times$ DSP and $0.78\times$ BRAM to achieve $4.5\times$ speedup. Compared with ARM, it could achieve $40.1\times$ and $39.8\times$ speedup on TUM and KITTI, respectively.

2) Accuracy analysis: We compare the feature point positions computed by our ORB extractor with the FAST extractor in OpenCV on KITTI dataset. In total, 96% locations of feature points extracted from FPGA accelerated FAST extractor match the locations extracted by OpenCV at CPU. The high accuracy of feature point extraction contributes to the overall end-to-end accuracy of the whole SLAM system.

To compare the accuracy of ac^{2}SLAM with eSLAM, the fr1/xyz, fr1/desk, fr1/room, fr2/xyz sequences in TUM are used for evaluation.

As shown in Fig. 7, benefited from the robust back-end in ORB-SLAM2, the absolute trajectory error of ac^{2}SLAM is 50% that of eSLAM on the fr1/xyz and fr1/desk sequences. On the fr2/xyz sequence, ac^{2}SLAM even reduces the absolute trajectory error to 10% of eSLAM.

Table V compares the frame rate of ac^{2}SLAM with the ARM processor. On the TUM and KITTI datasets, ac^{2}SLAM achieves $2.1\times$ and $2.7\times$ frame rate improvement, respectively.

TABLE V: Frame Rate

	TUM	KITTI
ac^{2}SLAM	15.5 fps	10.5 fps
ARM	7.5 fps	3.8 fps

TABLE VI: The Latency of ORB Extractor

	4 Parallelism	8 Parallelism	16 Parallelism
200 keypoints	1.3 ms	0.7 ms	0.4 ms
2000 keypoints	1.4 ms	1.0 ms	1.1 ms

3) Scalable design analysis: We use hardware-software co-simulation offered by Vivado-HLS to analyze the scalability of proposed ORB extractor with 4, 8 and 16 parallelism as well as the impact of sequential processing multiple image patches in RS-BRIEF, as discussed in section III-B0c.

As shown in Table VI, with 200 keypoints in the image, the latency reduction is roughly linear to the number of parallelism. When the image has thousands of keypoints, the sequential calculation in RS-BRIEF will become the bottleneck so that the latency cannot get further reduced by higher parallelism. Therefore, as for thousands of keypoints per image, parallel processing of image patches instead of sequential processing should be taken in RS-BRIEF to achieve throughput matching when parallelism is greater than 4.

B. PP-HEAPSORT Latency Evaluation

Fig. 8 shows the latency improvement of the proposed PP-HEAPSORT. All data are randomly generated and fed into the tested designs.

The end-to-end latency comparison between PP-HEAPSORT and sorting on ARM processor (embedded CPU on FPGA) is shown in Fig. 8a. The PP-HEAPSORT module achieves $1.9\times \sim 7.6\times$ when the input data size ranges from 1024 to 4096. The latency of sorting on ARM is proportional to the input data size. The latency of PP-HEAPSORT is almost stable, because the transmission delay of DMA (ms level) is much higher than the sorting delay of heapsort (μs level).

To further evaluate the pure latency on heapsort itself, we list the latency comparison with heapsort (II=2) [14], as shown in Fig. 8b. Compared with heapsort (II=2), the proposed PP-HEAPSORT achieves $1.94\times$ faster on average. The reason for not achieving exact $2\times$ is that data coordination between two sub-heaps through FIFOs takes extra time. Even though the latency introduced by data coordination will go up as the number of input data increasing, the comparison using random input sequence shows that the degradation is only 6% on average.

C. PP-HEAPSORT Resource Comparison

To quantify the resources saving of proposed PP-HEAPSORT, we compare an alternative design which could also achieve single-cycle initiation interval, i.e. heapsort implemented by LUT without BRAM. For giving a clear view of impact of the architecture, we synthesize both two designs under different bit-width of data. As shown in the Table VII, PP-HEAPSORT consumes only 14% LUT and 20% FF compared with LUT-based heapsort under both short and large data width. To make a conclusion, the PP-HEAPSORT amortizes the LUT/FF by LUT/FF/BRAM, so that LUT will not become the resource bottleneck. Thus the PP-HEAPSORT has better scalability than

TABLE VII: Utilization of Heap

	Data Width	LUT	FF	BRAM
PP-HEAPSORT	16	1808	1237	13
heapsort (II=1, LUT)	16	8100	4706	0
PP-HEAPSORT	292	29811	16694	205
heapsort (II=1, LUT)	292	206342	83069	0

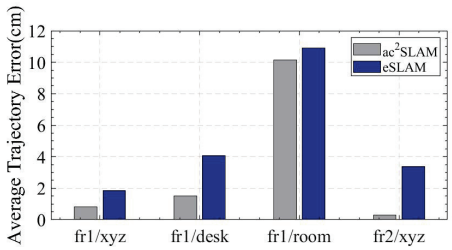
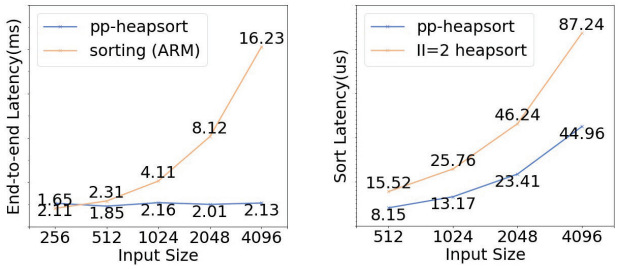


Fig. 7: Average Absolute Trajectory Error

TABLE VIII: Latency of parallel strategy

Unit 10^{-5}ms	BSys	SE
Serial	32.52	49.14
Parallel	21.21	31.50
parallel/serial	65.22%	64.10%



(a) end-to-end latency. (b) hardware latency.

Fig. 8: Latency comparison

heapsort with pure LUT as data storage i.e. PP-HEAPSORT with $2^n - 2$ size could achieve larger n compared with LUT-based heapsort with $2^n - 1$ size.

D. Back-end Parallel Experiment Analysis

1) The instruction of the experiment: The number of iteration in the BSys loop and the SE loop are n_e and n_l, respectively. We test the optimization of the back-end of the system on the board. The latency is evaluated on the KITTI datasets by averaging the results in multiple BA. As shown in Table VIII, the proposed parallel strategy only use 65.22% time of Bsys in ORB-SLAM2 [15], while 64.10% time of SE.

2) Latency reduction analysis: The number of both n_e and n_l are typically hundreds or thousands, and they become hundreds of thousands when detecting loop, as shown in Fig. 1. The larger number of n_e and n_l, the more calculations of the partial derivative and matrix multiplication in BSys and the more inversion and multiplication of matrices in SE. Multiple matrix calculations without output dependency can be distributed to different threads performed in parallel, which reduces the overall latency.

VII. Related Work

The ORB feature extractor consumes the most CPU resources in the front-end, which makes multiple previous works focus on ORB feature extractor acceleration. Fang et al. [5] implement an one-pixel-per-cycle ORB extracting accelerator based on FPGA, which reduces the latency by 51% and 41% compared with ARM Krait and Intel i5 CPU, respectively. Liu et al. [21] propose an FPGA accelerator for optical flow based front-end running in task-level-parallel fashion. eSLAM [4] proposes a FPGA-friendly rotational symmetric BRIEF (RS-BRIEF) descriptor and a SLAM system encompass FPGA-accelerated front-end and a simple back-end optimization running on CPU. Although these designs improve the performance of front-end and reschedule the workflow, they either ignore the optimization like [5], [21] or use a simple back-end optimization like [4]. The simplified optimization with only reprojection error of the observed map points minimizing can't achieve high accuracy in large fields with multiple loops.

Bundle adjustment including all flows in section V-A, is the main process of the back-end optimization, causing great latency in the ORB-SLAM2 system so many works focus on it. Jeong et al. [22] exploit efficient memory handling and fast block-based linear solving, and propose a novel embedded point iterations method to improve the BA performance on CPU but the three methods are not realized in parallel. Eriksson et al. [23] propose a distributed approach for very large scale global bundle adjustment computation. But the proposed method may be slower in datasets with small images.

VIII. Conclusion

In this paper, an FPGA-accelerated complete SLAM system based on ORB-SLAM2 is proposed to apply in real-time and ensure high accuracy. Firstly, the performance boosting in front-end comes from throughput matching of multiple heterogeneous modules. And the proposed design can also be easily tuned for larger deployment by synchronously expanding the parallelism of all modules. What's more, the PP-HEAPSORT offloads the keypoints selection from CPU to FPGA and thus not only reduces up to 90% transfer bandwidth but also achieves three-magnitude speed-up. The proposed multiple-heapsort coordination could also be scaled to larger heap with $2^n, (n = 1, 2, ...)$ sub-heaps. As for the back-end, the paper details the workflow of the back-end and concludes that the latency of back-end mainly lies at ten thousands loops of small matrix operations with less data reused. Leveraging all above three key-ideas, the ac^{2}SLAM achieves $4.55\times$ faster than previous SOTA eSLAM while $40\times$ faster than ARM deployment. The error of ac^{2}SLAM could achieve only 10% of the eSLAM. As we have raised a parallel strategy of back-end optimization and validate its feasibility on multi-core CPU, the further dedicated accelerator for the back-end optimization will be our future work.

Acknowledgments

This work was supported in part by the Key-Area Research and Development Program of Guangdong Province No. 2019B010153003 and the Fundamental Research Funds for the Xi'an Jiaotong University No.xhj032021005-05.

REFERENCES

[1] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *2011 International Conference on Computer Vision*, 2011, pp. 2564–2571.

[2] Y. Biadgie and K.-A. Sohn, "Feature detector using adaptive accelerated segment test," in *2014 International Conference on Information Science Applications (ICISA)*, 2014, pp. 1–4.

[3] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "Brief: Binary robust independent elementary features," in *Computer Vision – ECCV 2010*, K. Daniilidis, P. Maragos, and N. Paragios, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 778–792.

[4] R. Liu, J. Yang, Y. Chen, and W. Zhao, "Eslam: An energy-efficient accelerator for real-time orb-slam on fpga platform," in *Proceedings of the 56th Annual Design Automation Conference 2019*, ser. DAC '19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3316781.3317820

[5] W. Fang, Y. Zhang, B. Yu, and S. Liu, "Fpga-based orb feature extraction for real-time visual slam," in *2017 International Conference on Field Programmable Technology (ICFPT)*, 2017, pp. 275–278.

[6] Q. Gautier, A. Althoff, and R. Kastner, "Fpga architectures for real-time dense slam," in *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, vol. 2160-052X, 2019, pp. 83–90.

[7] J. Diaz, E. Ros, F. Pelayo, E. Ortigosa, and S. Mota, "Fpga-based real-time optical-flow system," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 16, no. 2, pp. 274–279, 2006.

[8] M. Komorkiewicz, T. Kryjak, K. Chuchacz-Kowalczyk, P. Skruch, and M. Gorgoń, "Fpga based system for real-time structure from motion computation," in *2015 Conference on Design and Architectures for Signal and Image Processing (DASIP)*, 2015, pp. 1–7.

[9] R. Mueller, J. Teubner, and G. Alonso, "Sorting networks on fpgas," *The VLDB Journal—The International Journal on Very Large Data Bases*, vol. 21, no. 1, pp. 1–23, 2012.

[10] D. Mihhailov, V. Sklyarov, I. Skliarova, and A. Sudnitson, "Parallel fpga-based implementation of recursive sorting algorithms," in *2010 International Conference on Reconfigurable Computing and FPGAs*. IEEE, 2010, pp. 121–126.

[11] V. Sklyarov, "Fpga-based implementation of recursive algorithms," *Microprocessors and Microsystems*, vol. 28, no. 5-6, pp. 197–211, 2004.

[12] W. J. Dally and B. P. Towles, *Principles and practices of interconnection networks*. Elsevier, 2004.

[13] C. M. Day Jr and J. N. Giacopelli, "Batcher-banyan network," Mar. 20 1990, uS Patent 4,910,730.

[14] W. Zabolotny, "Dual port memory based heapsort implementation for fpga," in *Symposium on Photonics Applications in Astronomy, Communications, Industry, and High-Energy Physics Experiments (WILGA)*, 2011.

[15] R. Mur-Artal and J. D. Tardós, "Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras," *IEEE transactions on robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[16] T. Lindeberg, *Scale-space theory in computer vision*. Springer Science & Business Media, 2013, vol. 256.

[17] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 573–580.

[18] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, "Vision meets robotics: The kitti dataset," *International Journal of Robotics Research (IJRR)*, 2013.

[19] D. Gálvez-López and J. D. Tardós, "Bags of binary words for fast place recognition in image sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, October 2012.

[20] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "G2o: A general framework for graph optimization," in *2011 IEEE International Conference on Robotics and Automation*, 2011, pp. 3607–3613.

[21] S. Liu, Z. Wan, B. Yu, and Y. Wang, "Robotic computing on fpgas," *Synthesis Lectures on Computer Architecture*, vol. 16, no. 1, pp. 1–218, 2021.

[22] J. Yekeun, N. David, S. Drew, S. Richard, and K. In-So, "Pushing the envelope of modern methods for bundle adjustment," *IEEE Trans.Pattern Anal.Mach.Intell*, vol. 34, no. 8, 2012.

[23] Z. Runze, Z. Siyu, S. Tianwei, Z. Lei, L. Zixin, F. Tian, and Q. Long, "Distributed very large scale bundle adjustment by global camera consensus," *IEEE Trans.Pattern Anal.Mach.Intell*, vol. 42, no. 2, 2020.